Graph Neural Networks for Credit Contagion

Modeling Systemic Risk Through Financial Networks

Andrés González Ortega

March 2026

Abstract

An applied treatment of Graph Neural Networks (GNNs) for modeling credit contagion in financial networks. Traditional credit risk models treat defaults as independent events or assume dependence arises only from common macroeconomic factors. The 2008 financial crisis demonstrated that interconnectedness itself is a source of systemic risk: defaults cascade through networks of financial obligations. We develop the framework from first principles: why network structure matters for credit risk; how financial networks are represented as directed, weighted graphs; the mechanisms of direct, fire-sale, and funding contagion; how GNNs learn node-level representations by aggregating neighborhood information; Graph Convolutional Networks (GCNs) for default prediction; the Eisenberg–Noe clearing mechanism for generating cascade labels; and systemic risk metrics including DebtRank, CoVaR, and SRISK. A worked example runs throughout: a 50-node interbank network where GCN layers propagate risk signals across the graph to predict which institutions default following an exogenous shock. We assume familiarity with neural networks, basic graph theory, and credit risk fundamentals.

Contents

1 Why Networks Matter for Credit Risk **3**
1.1 The Independence Assumption and Its Failure . . . 3
1.2 What Networks Add . . . 3
1.3 The Contribution of This Project . . . 4

2 Financial Networks **4**
2.1 Types of Financial Linkages . . . 4
2.2 The Liability Matrix . . . 5
2.3 Topological Metrics . . . 5
2.4 Node Features . . . 6

3 Contagion Mechanisms **6**
3.1 Direct (Credit) Contagion . . . 6
3.1.1 The Eisenberg–Noe Clearing Mechanism . . . 7
3.2 Fire-Sale Contagion . . . 8
3.3 Funding (Liquidity) Contagion . . . 8

4 Graph Neural Networks: The Key Idea **9**
4.1 Why GNNs Fit Contagion . . . 9
4.2 The Message-Passing Framework . . . 9
4.3 Key Architectures . . . 10

5 GCN for Default Prediction **11**
5.1 Problem Formulation 11
5.2 Two-Layer GCN Architecture 11
5.3 What the Normalization Does 11
5.4 Loss Function 12

6 Cascade Simulation **12**
6.1 Simulation Procedure 13
6.2 Shock Design 13

7 Systemic Risk Metrics **14**
7.1 DebtRank 14
7.2 CoVaR 14
7.3 SRISK 15
7.4 Using the GNN to Compute Systemic Metrics 15

8 References **16**

1 Why Networks Matter for Credit Risk

Consider a stylized banking system with three banks: A, B, and C. Bank A has lent \$500 million to Bank B, and Bank B has lent \$400 million to Bank C. An exogenous shock wipes out Bank C's assets. Bank C defaults. Bank B now loses \$400 million — if this exceeds B's capital buffer, B defaults too. Bank A then loses its \$500 million exposure to B, and the cascade continues.

This is **credit contagion**: the propagation of financial distress through a network of contractual obligations.

1.1 The Independence Assumption and Its Failure

Traditional credit risk models rest on assumptions about how defaults are correlated:

- **Merton (1974)**: each firm defaults when its asset value falls below its liabilities. Correlations arise from shared exposure to systematic factors (market returns). But the model treats each firm as an isolated entity — it does not account for the contractual links between them.
- **CreditRisk+ (Credit Suisse, 1997)**: defaults are driven by independent Poisson processes, modulated by common risk factors. Dependencies come from shared factors, not from direct financial linkages.
- **Gaussian copula models**: a correlation parameter ρ controls joint default probabilities, but the correlation is uniform and does not reflect the *topology* of who owes whom.

These models capture the idea that a recession can cause many defaults simultaneously. What they miss is that defaults cause other defaults *mechanically*: through the network of exposures.

Warning

The 2008 financial crisis provided a devastating empirical demonstration. Lehman Brothers filed for bankruptcy on 15 September 2008 with approximately 930,000 derivative contracts outstanding. Its default did not just cause losses at its direct counterparties — it triggered a chain reaction. Counterparties that lost money on Lehman exposures became weaker, their creditors became nervous, liquidity evaporated, and asset prices collapsed. The total systemic losses far exceeded the direct losses on Lehman exposures. **Interconnectedness amplified a single default into a global crisis.**

1.2 What Networks Add

A network representation captures what traditional models cannot:

1. **Direct linkages**: who owes money to whom, and how much.
2. **Concentration risk**: if many institutions are exposed to the same counterparty, a single default can trigger widespread losses.
3. **Cascade dynamics**: a default at node i reduces the assets of node i's creditors, potentially causing them to default, which in turn affects *their* creditors.
4. **Topology matters**: a highly connected "hub" bank failing is far more dangerous than a peripheral institution failing. Network centrality measures quantify this.

Core Idea

Credit risk is not just about individual balance sheets. It is about the **structure of obligations** between institutions. Two banking systems with identical individual bank characteristics but different network topologies can have dramatically different systemic risk profiles. A network with a single highly connected hub is more fragile than a diversified network with many moderate connections.

1.3 The Contribution of This Project

We combine two powerful ideas:

1. **Financial network models** (Eisenberg–Noe, DebtRank) that simulate how shocks cascade through interbank exposures.
2. **Graph Neural Networks** that learn to predict which institutions will default by aggregating information across the network topology.

The GNN approach is attractive because it can learn *non-linear* contagion patterns that simple cascade models may miss, and it can generalize to networks with different topologies once trained.

2 Financial Networks

A financial network is a directed, weighted graph $G = (V, E, W)$ where:

- $V = \{1, 2, \ldots, n\}$ is the set of **nodes** (institutions: banks, insurance companies, hedge funds, corporates).
- $E \subseteq V \times V$ is the set of **directed edges** representing financial obligations.
- $W : E \to \mathbb{R}_{>0}$ assigns a **weight** w_{ij} to each edge (i, j), representing the nominal value of i's obligation to j (equivalently, j's exposure to i).

2.1 Types of Financial Linkages

Multiple types of edges can coexist in a financial network:

Linkage Type	Edge Meaning	Example
Interbank lending	Bank i has borrowed from bank j	Overnight lending, term deposits
Derivative exposures	Bank i owes mark-to-market to j	Interest rate swaps, CDS contracts
Payment systems	Bank i sends payments through j	Fedwire, TARGET2
Supply chain	Firm i supplies goods to firm j	Trade credit, accounts receivable
Common asset holdings	i and j hold overlapping portfolios	Indirect link via fire-sale channel

2.2 The Liability Matrix

The network is compactly represented by an $n \times n$ **liability matrix** L, where L_{ij} is the total obligation of institution i to institution j:

$$L = \begin{pmatrix} 0 & L_{12} & \cdots & L_{1n} \\ L_{21} & 0 & \cdots & L_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ L_{n1} & L_{n2} & \cdots & 0 \end{pmatrix} \tag{1}$$

The diagonal is zero (no self-obligations). The total interbank liabilities of node i are $\bar{p}_i = \sum_{j=1}^{n} L_{ij}$, and the total interbank assets of node i are $\sum_{j=1}^{n} L_{ji}$.

Each institution also has **external assets** e_i (assets not related to interbank claims) and **external liabilities** d_i (deposits, bonds, obligations to non-bank creditors).

2.3 Topological Metrics

Network topology determines how shocks propagate. The following centrality measures quantify the importance of each node:

Definition 2.1 (Centrality Measures)**.** For a weighted directed graph with adjacency matrix A (where $A_{ij} = w_{ij}$ if edge (i, j) exists, 0 otherwise):

1. **Degree centrality**: the number of connections. In-degree $k_i^{\text{in}} = \sum_j A_{ji}$ counts incoming edges (exposures *to* i). Out-degree $k_i^{\text{out}} = \sum_j A_{ij}$ counts outgoing edges (i's obligations).

2. **Betweenness centrality**: the fraction of shortest paths between all pairs of nodes that pass through node i. A high-betweenness node is a critical intermediary; its failure disrupts many channels.

3. **Eigenvector centrality**: solves $Ax = \lambda x$. A node is important if it is connected to other important nodes. The eigenvector corresponding to the largest eigenvalue assigns a centrality score to each node.

Example 2.1 (Core-periphery structure)**.** Empirical interbank networks consistently exhibit **core-periphery structure**:

- A small **core** of 10–15 major banks that are densely connected to each other (high degree, high betweenness).

- A large **periphery** of smaller institutions that connect to one or two core banks but not to each other.

This structure has important implications for contagion:

- A core bank failing can propagate shocks to many peripheral banks simultaneously.

- A peripheral bank failing typically affects only one or two core banks, which can usually absorb the loss.

- The system is robust to random peripheral failures but fragile to targeted core failures — the "robust-yet-fragile" property (Acemoglu, Ozdaglar & Tahbaz-Salehi, 2015).

2.4 Node Features

Beyond topology, each node carries **features** that determine its resilience to shocks. For a GNN, these form the initial node feature matrix $X \in \mathbb{R}^{n \times d}$ where d is the number of features per node:

Feature	Symbol	Interpretation
Capital ratio	x_1	Equity / total assets; buffer against losses
Leverage	x_2	Total assets / equity; amplifies shocks
Asset quality	x_3	Non-performing loans / total loans
Size (log assets)	x_4	Systemic importance proxy
Liquidity ratio	x_5	Liquid assets / short-term liabilities

A bank with a high capital ratio and low leverage can absorb larger losses before defaulting. The GNN will learn to combine these features with the network structure.

Key Insight

A financial network is fully described by three components:

1. **Topology** — the adjacency/liability matrix L: who owes whom, how much.
2. **Node features** — the feature matrix X: balance sheet characteristics of each institution.
3. **External variables** — external assets e_i and liabilities d_i that anchor each institution's solvency outside the network.

The GNN operates on all three simultaneously.

3 Contagion Mechanisms

Financial contagion operates through several distinct channels. Understanding these channels is essential for designing the cascade simulation that generates training labels for the GNN.

3.1 Direct (Credit) Contagion

The most transparent mechanism: institution A defaults on its obligations, and institution B, which is owed money by A, suffers a loss equal to its exposure (net of recovery).

Formally, if institution i defaults and j has an exposure L_{ij} to i with recovery rate R, then j's loss is:

$$\ell_j = (1 - R) \cdot L_{ij} \tag{2}$$

If ℓ_j exceeds j's capital buffer c_j, then j defaults as well, and the cascade continues.

Core Idea

Direct contagion is a **domino effect**. The key insight from Eisenberg & Noe (2001) is that we cannot simply iterate default-by-default, because obligations are *simultaneous*: when i defaults, it may only partially pay its creditors, and the partial payment depends on how much i receives from *its* debtors, who may themselves be defaulting. This circular dependency requires solving for a **clearing payment vector** — a fixed-point problem.

3.1.1 The Eisenberg–Noe Clearing Mechanism

Consider n institutions with interbank liability matrix L. Let $\bar{p}_i = \sum_j L_{ij}$ be the total interbank obligations of node i, and define the **relative liability matrix**:

$$\Pi_{ij} = \frac{L_{ij}}{\bar{p}_i} \quad \text{for } \bar{p}_i > 0 \tag{3}$$

so that Π_{ij} is the fraction of i's total interbank obligations owed to j.

Each institution i has external assets e_i (cash, securities, loans to the real economy). The total resources available to i are its external assets plus whatever it receives from other institutions:

$$a_i(p) = e_i + \sum_{j=1}^{n} \Pi_{ji}\, p_j \tag{4}$$

where p_j is the actual payment made by institution j.

Definition 3.1 (Clearing Payment Vector)**.** A payment vector $p^* = (p_1^*, \ldots, p_n^*)$ is a **clearing vector** if for each institution i:

$$p_i^* = \min\big(\bar{p}_i,\ a_i(p^*)\big) = \min\left(\bar{p}_i,\ e_i + \sum_{j=1}^{n} \Pi_{ji}\, p_j^*\right) \tag{5}$$

That is, each institution pays the minimum of what it owes ($\bar{p}_i$) and what it has (a_i). If $a_i(p^*) < \bar{p}_i$, institution i is in **default** and pays pro rata to its creditors.

Theorem 3.1 (Eisenberg & Noe, 2001)**.** *Under mild regularity conditions (the financial system has no cycles of institutions with zero external assets), the clearing payment vector p^* exists and is unique. It can be computed by the following **fixed-point iteration** ("fictitious default algorithm"):*

1. *Initialize $p^{(0)} = \bar{p}$ (every institution pays in full).*
2. *At each iteration k, compute:*

$$p_i^{(k+1)} = \min\left(\bar{p}_i,\ e_i + \sum_{j=1}^{n} \Pi_{ji}\, p_j^{(k)}\right)$$

3. *Repeat until convergence: $\|p^{(k+1)} - p^{(k)}\| < \varepsilon$.*

The sequence $p^{(0)} \geq p^{(1)} \geq \cdots$ is non-increasing and converges to p^* in at most n iterations (each iteration identifies at least one new defaulting institution).

Example 3.1 (Three-bank clearing)**.** Three banks with liabilities:

$$L = \begin{pmatrix} 0 & 80 & 0 \\ 0 & 0 & 60 \\ 40 & 0 & 0 \end{pmatrix}$$

So bank 1 owes 80 to bank 2, bank 2 owes 60 to bank 3, bank 3 owes 40 to bank 1. External assets: $e_1 = 100$, $e_2 = 50$, $e_3 = 30$.

No shock: total assets of bank 1 $= e_1 +$ received from bank 3 $= 100 + 40 = 140 > 80$ (pays in full). Bank 2 $= 50 + 80 = 130 > 60$ (pays in full). Bank 3 $= 30 + 60 = 90 > 40$ (pays in full). No defaults.

Shock: reduce e_1 to 20 (asset loss of 80). Now bank 1's total assets $= 20 + 40 = 60 < 80$ (default; pays 60 pro rata to bank 2). Bank 2 receives 60 instead of 80; total assets $= 50 + 60 = 110 > 60$ (survives). Bank 3 receives 60 from bank 2; total assets $= 30 + 60 = 90 > 40$ (survives). **One default**, no cascade.

Larger shock: reduce e_1 to 0. Bank 1's total assets $= 0 + 40 = 40 < 80$ (default; pays 40). Bank 2 receives 40; total assets $= 50 + 40 = 90 > 60$ (survives). Still only one default, because bank 2 is well-capitalized.

3.2 Fire-Sale Contagion

When a distressed institution is forced to sell illiquid assets quickly, the sales depress market prices. Other institutions holding the same assets suffer mark-to-market losses even though they are not directly connected to the distressed institution.

Let Δq_k be the quantity of asset k sold by the distressed institution. If the price impact function is linear, the new price is:

$$p'_k = p_k \left(1 - \frac{\Delta q_k}{\mathrm{ADV}_k}\right) \tag{6}$$

where ADV_k is the average daily volume of asset k. All institutions holding asset k suffer a loss proportional to their holdings.

Warning

Fire-sale contagion is particularly dangerous because it operates through **indirect linkages**: two institutions that have no direct contractual relationship can infect each other if they hold overlapping portfolios. This channel was central to the 2008 crisis, when forced selling of mortgage-backed securities caused prices to spiral downward, inflicting losses on institutions far removed from the original subprime defaults.

3.3 Funding (Liquidity) Contagion

When an institution is perceived as risky, its funding counterparties may:

- Refuse to roll over short-term funding (commercial paper, repo).
- Demand higher collateral (margin calls).
- Withdraw deposits.

This creates a self-fulfilling prophecy: the institution's liquidity position deteriorates, forcing asset sales (fire-sale channel) or outright default, even if it is fundamentally solvent. Northern Rock (2007) and Bear Stearns (2008) exemplify pure funding contagion.

Key Insight

The three contagion channels interact and amplify each other:

1. **Direct contagion** causes credit losses at counterparties.
2. Weakened counterparties are forced to sell assets (**fire-sale contagion**).
3. Market participants observe distress and withdraw funding (**funding contagion**).
4. Funding withdrawal forces more asset sales, depressing prices further.

For our GNN implementation, we focus on direct contagion via the Eisenberg–Noe mechanism, as it is the most natural fit for graph-structured data. The GNN's message-passing layers can implicitly learn patterns that echo fire-sale and funding dynamics through shared features.

4 Graph Neural Networks: The Key Idea

A Graph Neural Network learns representations of nodes (or edges, or the entire graph) by **iteratively aggregating information from neighboring nodes**. At each layer, a node's representation is updated based on its own features *and* the features of its neighbors.

4.1 Why GNNs Fit Contagion

Consider why this architecture is a natural match for credit contagion:

- The risk of institution i depends on the health of institutions *connected to* i (its direct counterparties).
- But it also depends on the health of institutions connected to i's counterparties (indirect exposure, two hops away).
- And so on, for longer chains of exposure.

A GNN with k layers captures exactly k-hop neighborhood information. After one layer, each node "knows" about its direct neighbors. After two layers, each node has incorporated information from two-hop neighbors. This mirrors how contagion propagates: one round of defaults, then second-round effects, then third-round effects.

Core Idea

The message-passing mechanism in a GNN is a **learned analogue of cascade simulation**. Where the Eisenberg–Noe model uses a fixed rule ("pay what you can"), the GNN learns a parametric aggregation function from data. This allows it to capture non-linear interactions, heterogeneous node responses, and patterns that a simple threshold model may miss.

4.2 The Message-Passing Framework

All GNN architectures follow the same abstract **message-passing** pattern. For each node i at layer l:

$$h_i^{(l+1)} = \phi\left(h_i^{(l)}, \bigoplus_{j \in \mathcal{N}(i)} \psi\left(h_i^{(l)}, h_j^{(l)}, e_{ij}\right)\right) \tag{7}$$

where:

- $h_i^{(l)} \in \mathbb{R}^{d_l}$ is the **embedding** (hidden representation) of node i at layer l. At layer 0, $h_i^{(0)} = x_i$ (the raw node features).
- $\mathcal{N}(i)$ is the set of **neighbors** of node i.
- $\psi(\cdot)$ is the **message function**: it computes a "message" from neighbor j to node i, potentially using the edge weight e_{ij}.
- $\bigoplus$ is an **aggregation operator** (sum, mean, max) that combines messages from all neighbors into a single vector.
- $\phi(\cdot)$ is the **update function**: it combines the aggregated messages with the node's current embedding to produce the new embedding.

Different GNN architectures correspond to different choices of ψ, $\bigoplus$, and ϕ.

4.3 Key Architectures

Definition 4.1 (GCN — Kipf & Welling, 2017)**.** The Graph Convolutional Network uses a simple spectral-inspired aggregation:

$$H^{(l+1)} = \sigma\Big(\tilde{D}^{-1/2}\,\tilde{A}\,\tilde{D}^{-1/2}\,H^{(l)}\,W^{(l)}\Big) \tag{8}$$

where $\tilde{A} = A + I$ (adjacency matrix with added self-loops), $\tilde{D}$ is the degree matrix of $\tilde{A}$ ($\tilde{D}_{ii} = \sum_j \tilde{A}_{ij}$), $H^{(l)} \in \mathbb{R}^{n \times d_l}$ is the node feature matrix at layer l, $W^{(l)} \in \mathbb{R}^{d_l \times d_{l+1}}$ is a learnable weight matrix, and σ is a non-linear activation function (typically ReLU).

Definition 4.2 (GraphSAGE — Hamilton et al., 2017)**.** GraphSAGE samples a fixed number of neighbors and learns an aggregation function:

$$h_{\mathcal{N}(i)}^{(l)} = \text{AGGREGATE}\Big(\{h_j^{(l)} : j \in \mathcal{N}(i)\}\Big) \tag{9}$$

$$h_i^{(l+1)} = \sigma\Big(W^{(l)} \cdot \big[h_i^{(l)} \,\|\, h_{\mathcal{N}(i)}^{(l)}\big]\Big) \tag{10}$$

where $\|$ denotes concatenation and AGGREGATE can be mean, LSTM, or max-pooling. The concatenation of self-features with neighbor-features explicitly separates the node's own information from its neighborhood signal.

Definition 4.3 (GAT — Veličković et al., 2018)**.** The Graph Attention Network learns attention weights that determine how much each neighbor contributes:

$$\alpha_{ij} = \frac{\exp\big(\text{LeakyReLU}(a^T[Wh_i \,\|\, Wh_j])\big)}{\sum_{k \in \mathcal{N}(i)} \exp\big(\text{LeakyReLU}(a^T[Wh_i \,\|\, Wh_k])\big)} \tag{11}$$

$$h_i^{(l+1)} = \sigma\left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij}\, W\, h_j^{(l)}\right) \tag{12}$$

where a is a learnable attention vector. The attention weights α_{ij} allow the model to focus on the most relevant neighbors — for credit contagion, it might learn to pay more attention to highly leveraged counterparties.

Remark 4.1. For credit contagion, GAT is conceptually appealing because not all counterparties are equally dangerous. A highly leveraged neighbor is a greater contagion risk than a well-capitalized one. Attention weights can learn this distinction from data without explicit feature engineering.

5 GCN for Default Prediction

We now develop the specific GCN architecture for predicting which nodes in a financial network will default following an exogenous shock.

5.1 Problem Formulation

Given:

- A financial network $G = (V, E)$ with n nodes.
- Adjacency matrix $A \in \mathbb{R}^{n \times n}$ (derived from the liability matrix; $A_{ij} = 1$ if $L_{ij} > 0$, or $A_{ij} = L_{ij}$ for a weighted graph).
- Node feature matrix $X \in \mathbb{R}^{n \times d}$ with d features per node (capital ratio, leverage, asset quality, size, liquidity ratio).
- A shock scenario s that reduces the external assets of one or more nodes.

Predict: for each node i, the probability $\hat{y}_i \in [0, 1]$ that node i defaults under shock scenario s.

This is a **node-level binary classification** task on a graph.

5.2 Two-Layer GCN Architecture

We use a two-layer GCN, which captures information from 2-hop neighborhoods:

Layer 1 (feature extraction):

$$H^{(1)} = \mathrm{ReLU}\Big(\tilde{D}^{-1/2}\,\tilde{A}\,\tilde{D}^{-1/2}\,X\,W^{(0)}\Big) \tag{13}$$

This maps from the original $d = 5$ features to a hidden dimension $d_1 = 16$. So $W^{(0)} \in \mathbb{R}^{5 \times 16}$.

Layer 2 (classification):

$$Z = \mathrm{sigmoid}\Big(\tilde{D}^{-1/2}\,\tilde{A}\,\tilde{D}^{-1/2}\,H^{(1)}\,W^{(1)}\Big) \tag{14}$$

This maps from the hidden dimension $d_1 = 16$ to a single output. So $W^{(1)} \in \mathbb{R}^{16 \times 1}$, and $Z \in \mathbb{R}^{n \times 1}$ contains the predicted default probabilities.

The sigmoid activation in the final layer ensures outputs are in $[0, 1]$.

5.3 What the Normalization Does

The symmetric normalization $\tilde{D}^{-1/2}\,\tilde{A}\,\tilde{D}^{-1/2}$ has an intuitive interpretation. For each node i, it computes a weighted average of its neighbors' features (including its own, via the self-loop in $\tilde{A}$):

$$h_i^{(l+1)} = \sigma\left(\sum_{j \in \mathcal{N}(i) \cup \{i\}} \frac{1}{\sqrt{\tilde{d}_i\,\tilde{d}_j}}\,h_j^{(l)}\,W^{(l)}\right) \tag{15}$$

where $\tilde{d}_i = 1 + \sum_j A_{ij}$ is the degree of node i in the augmented graph. The $1/\sqrt{\tilde{d}_i\,\tilde{d}_j}$ factor normalizes by the degrees of both the sending and receiving nodes, preventing high-degree nodes from dominating the aggregation.

Example 5.1 (Worked example: 50-node network)**.** Consider a synthetic interbank network with $n = 50$ nodes and $d = 5$ features per node.

Network: generated with a core-periphery structure. 10 core banks are densely connected (edge probability 0.7 within core). 40 peripheral banks each connect to 1–3 core banks. Total edges: approximately 100.

Features: for each node, we generate $x_i \in \mathbb{R}^5$: capital ratio (mean 0.08, std 0.03), leverage (mean 15, std 5), NPL ratio (mean 0.03, std 0.02), log-assets (mean 10, std 2), liquidity ratio (mean 0.25, std 0.10).

Architecture:

- Input: $X \in \mathbb{R}^{50\times 5}$.
- Layer 1: $H^{(1)} = \mathrm{ReLU}(\hat{A}\, X\, W^{(0)}) \in \mathbb{R}^{50\times 16}$, where $\hat{A} = \tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}$.
- Layer 2: $Z = \mathrm{sigmoid}(\hat{A}\, H^{(1)}\, W^{(1)}) \in \mathbb{R}^{50\times 1}$.

Parameters: $W^{(0)}$ has $5 \times 16 = 80$ entries, $W^{(1)}$ has $16 \times 1 = 16$ entries. Total learnable parameters: **96**. This is remarkably compact — the network structure provides the inductive bias that keeps the model small.

Training: we generate 1,000 shock scenarios using the Eisenberg–Noe cascade simulation (Section 6), producing binary default labels for each node under each scenario. Train with binary cross-entropy loss and the Adam optimizer.

5.4 Loss Function

The binary cross-entropy loss for node-level classification:

$$\mathcal{L} = -\frac{1}{n}\sum_{i=1}^{n}\Big[y_i \log \hat{y}_i + (1 - y_i)\log(1 - \hat{y}_i)\Big] \tag{16}$$

where $y_i \in \{0, 1\}$ is the true default label (from the cascade simulation) and $\hat{y}_i = Z_i$ is the GCN's predicted default probability.

Warning

Default events are typically **rare**: in a 50-node network, a typical shock might cause 2–5 defaults. This means the dataset is heavily imbalanced (90%+ non-default labels). Standard remedies apply:

- **Weighted loss**: increase the weight on the positive (default) class.
- **Oversampling**: generate more severe shock scenarios to increase the default rate.
- **Focal loss**: down-weight easy (non-default) examples.

Without addressing class imbalance, the GCN may learn to predict "no default" for every node and achieve high accuracy but zero recall.

6 Cascade Simulation

The cascade simulation generates the **training data** for the GNN. We simulate the Eisenberg–Noe clearing mechanism under many shock scenarios to produce binary default labels for each node.

6.1 Simulation Procedure

1. **Generate network**: create an n-node financial network with realistic core-periphery topology. Assign edge weights (interbank liabilities) and node features (capital ratio, leverage, etc.). Assign external assets e_i to each node.

2. **Sample shock**: select one or more "seed" nodes to shock. Reduce their external assets by a fraction $\delta \in [0, 1]$:
$$e_i^{\text{shocked}} = (1 - \delta)\, e_i \tag{17}$$
Vary δ and the choice of seed nodes across scenarios.

3. **Run clearing**: apply the Eisenberg–Noe fixed-point iteration (Theorem 3.1) to find the clearing payment vector p^*.

4. **Identify defaults**: institution i is in default if $p_i^* < \bar{p}_i$ (it cannot pay its obligations in full). Assign label $y_i = 1$ (default) or $y_i = 0$ (solvent).

5. **Record**: store the tuple (X, A, y) where X is the (potentially shocked) node feature matrix, A is the adjacency matrix, and $y \in \{0, 1\}^n$ is the vector of default labels.

6. **Repeat**: generate M scenarios (e.g., $M = 1{,}000$) by varying the shock location and severity.

6.2 Shock Design

The choice of shocks determines the diversity and difficulty of the training data:

Shock Type	**Description**	**Purpose**
Single-node	Shock one randomly chosen node	Tests direct contagion from each node
Multi-node	Shock 2–5 randomly chosen nodes	Tests overlapping cascades
Targeted core	Shock a high-centrality core node	Tests worst-case systemic scenarios
Severity sweep	Fix the shocked node, vary δ from 0.2 to 1.0	Maps the contagion threshold

Example 6.1 (Cascade statistics)**.** For a 50-node core-periphery network with 10 core banks and 40 peripheral banks:

Shock scenario	**Avg. defaults**	**Max defaults**
Single peripheral, $\delta = 0.5$	1.2	3
Single peripheral, $\delta = 1.0$	1.8	5
Single core, $\delta = 0.5$	3.4	8
Single core, $\delta = 1.0$	6.1	15
Two core banks, $\delta = 0.8$	9.3	22

Observation: shocking a core bank produces far more cascading defaults than shocking a peripheral bank at the same severity. The network topology amplifies shocks that hit the core.

Key Insight

The cascade simulation serves two purposes:

1. **Training data**: the (X, A, y) tuples are used to train the GCN to predict defaults from node features and network topology.
2. **Benchmark**: the clearing mechanism provides ground-truth default labels against which the GCN's predictions are evaluated. If the GCN can learn to predict cascade outcomes accurately, it can be used as a **fast surrogate** for the computationally expensive clearing simulation — enabling real-time systemic risk assessment.

7 Systemic Risk Metrics

Once we can simulate cascades (or predict them with a trained GNN), we can compute metrics that quantify the systemic importance of each institution and the overall fragility of the network.

7.1 DebtRank

DebtRank (Battiston et al., 2012) measures the fraction of total economic value in the network that is lost when a given institution enters distress.

Definition 7.1 (DebtRank)**.** Let v_i denote the total economic value (equity + interbank assets) of institution i, and let $V = \sum_i v_i$ be the total system value. When institution i is shocked into distress, the cascade propagates and causes losses at other institutions. Let L_i^{total} be the total loss across the system (including at i itself). The DebtRank of institution i is:

$$\text{DebtRank}(i) = \frac{L_i^{\text{total}} - v_i}{V - v_i} \tag{18}$$

This is the fraction of **other institutions'** value lost due to the distress of i. A DebtRank of 0 means i's failure has no systemic impact. A DebtRank of 1 means i's failure wipes out the entire rest of the system.

The DebtRank computation uses a *linear contagion* model: when institution j is exposed to distressed institution i, j's equity is reduced proportionally to the exposure:

$$h_j^{(t+1)} = \min\left(1,\ h_j^{(t)} + \sum_{i:\, s_i^{(t)} = \text{active}} W_{ji}\, h_i^{(t)}\right) \tag{19}$$

where $h_i^{(t)} \in [0, 1]$ is the "distress level" of institution i at round t, $W_{ji} = L_{ij}/v_j$ is the relative exposure of j to i, and $s_i^{(t)}$ tracks whether i is still actively propagating distress.

Remark 7.1. DebtRank differs from the Eisenberg–Noe model in a crucial way: it uses **continuous distress levels** $h_i \in [0, 1]$ rather than binary default. This captures the economic damage from a bank's stock price declining by 50% (partial distress) even if it does not technically default. It is therefore a more sensitive measure of systemic impact.

7.2 CoVaR

CoVaR (Adrian & Brunnermeier, 2016) measures the tail risk of the financial system conditional on the distress of a specific institution.

Definition 7.2 (CoVaR)**.** The $\mathrm{CoVaR}_q^{j|i}$ is the q-quantile of the loss distribution of institution (or system) j, **conditional** on institution i being in distress (typically defined as i's return being at its VaR_q level):

$$\mathbb{P}\Big(X_j \leq \mathrm{CoVaR}_q^{j|i} \;\Big|\; X_i = \mathrm{VaR}_q^i\Big) = q \tag{20}$$

The systemic risk contribution of institution i is:

$$\Delta\,\mathrm{CoVaR}_q^{j|i} = \mathrm{CoVaR}_q^{j|i} - \mathrm{CoVaR}_q^{j|\mathrm{median}(i)} \tag{21}$$

This measures how much worse the system's tail risk becomes when institution i is in distress compared to when i is at its median state.

In our network context, X_j can be the total system loss (sum of losses across all institutions), and we condition on institution i being at or beyond its VaR threshold. The cascade simulation naturally produces these conditional distributions: run the cascade with i shocked, record system-wide losses, and compute quantiles.

7.3 SRISK

SRISK measures the expected capital shortfall of an institution in a systemic crisis.

Definition 7.3 (SRISK)**.**

$$\mathrm{SRISK}_i = \mathbb{E}\big[\max\big(k \cdot A_i - E_i,\ 0\big) \;\big|\; \text{crisis}\big] \tag{22}$$

where k is a prudential capital ratio (e.g., 8%), A_i is the total assets of institution i under the crisis scenario, and E_i is its equity. $\mathrm{SRISK}_i > 0$ means institution i would need a capital injection in a crisis. The total system SRISK, $\sum_i \mathrm{SRISK}_i$, is a measure of the system's overall capital shortfall and hence its vulnerability.

7.4 Using the GNN to Compute Systemic Metrics

Once the GCN is trained, it can serve as a fast proxy for the cascade simulation:

1. **DebtRank via GNN**: for each node i, create a shock scenario (reduce i's capital), run the GCN to predict default probabilities for all other nodes, and compute the expected system-wide loss. This is orders of magnitude faster than running the full Eisenberg–Noe clearing for each node.

2. **CoVaR via GNN**: generate many shock scenarios, use the GCN to predict default outcomes, and compute the conditional VaR of system losses given distress at node i.

3. **Stress testing**: apply a macroeconomic shock scenario (e.g., all banks' external assets decrease by 20%), feed the shocked features into the GCN, and obtain a real-time map of which institutions are most likely to default.

Core Idea

The GNN is not just a prediction model — it is a **computational accelerator** for systemic risk analysis. Full cascade simulations on large networks (thousands of nodes) are expensive. A trained GNN performs the same prediction in a single forward pass (two matrix multiplications), enabling real-time scenario analysis, interactive stress testing, and optimization of network structure for resilience.

Go Deeper

- For a rigorous treatment of DebtRank and its extensions, see Bardoscia et al. (2017), "Pathways towards instability in financial networks," *Nature Communications* 8, 14416.
- For the connection between GNNs and financial stability analysis, see the survey by Xu et al. (2021), "Graph Neural Networks for Financial Network Analysis."
- For the theoretical foundations of network contagion, see Acemoglu, Ozdaglar & Tahbaz-Salehi (2015), "Systemic Risk and Stability in Financial Networks," *American Economic Review* 105(2), 564–608.

8 References

The following works provide the theoretical foundations for the models and methods developed in this document.

[1] **Eisenberg, L. & Noe, T.H.** (2001). "Systemic Risk in Financial Systems." *Management Science*, 47(2), 236–249. The foundational clearing mechanism for interbank payment systems. Proves existence and uniqueness of the clearing payment vector and establishes the fixed-point algorithm used in Section 6.

[2] **Battiston, S., Puliga, M., Kaushik, R., Tasca, P. & Caldarelli, G.** (2012). "DebtRank: Too Central to Fail? Financial Networks, the FED and Systemic Risk." *Scientific Reports*, 2, 541. Introduces the DebtRank metric and applies it to the network of emergency lending during the 2008 crisis, showing that the largest banks also had the highest systemic impact.

[3] **Kipf, T.N. & Welling, M.** (2017). "Semi-Supervised Classification with Graph Convolutional Networks." *Proceedings of ICLR 2017*. The seminal GCN paper. Derives the spectral motivation for the $\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}$ normalization and demonstrates its effectiveness on citation networks.

[4] **Adrian, T. & Brunnermeier, M.K.** (2016). "CoVaR." *American Economic Review*, 106(7), 1705–1741. Defines CoVaR and ΔCoVaR as systemic risk measures and estimates them for major financial institutions using quantile regression.

[5] **Acemoglu, D., Ozdaglar, A. & Tahbaz-Salehi, A.** (2015). "Systemic Risk and Stability in Financial Networks." *American Economic Review*, 105(2), 564–608. Shows that the relationship between connectivity and stability is non-monotone: moderate diversification enhances stability, but beyond a threshold, more connections propagate shocks more widely.

[6] **Hamilton, W.L., Ying, R. & Leskovec, J.** (2017). "Inductive Representation Learning on Large Graphs." *Proceedings of NeurIPS 2017*. Introduces GraphSAGE, a scalable GNN framework that samples and aggregates neighborhood features.

[7] **Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P. & Bengio, Y.** (2018). "Graph Attention Networks." *Proceedings of ICLR 2018*. Introduces attention-based aggregation for GNNs, enabling learned weighting of neighbor contributions.

[8] **Bardoscia, M., Battiston, S., Caccioli, F. & Caldarelli, G.** (2017). "Pathways towards instability in financial networks." *Nature Communications*, 8, 14416. Extends DebtRank to incorporate multiple contagion channels and shows how small shocks can trigger large cascades through non-linear amplification.

Read Before Coding

Before implementing the GCN for credit contagion in code, ensure you are comfortable with:

- Basic graph theory: adjacency matrices, degree, paths, connected components.
- Neural network fundamentals: forward pass, backpropagation, loss functions, gradient descent.
- PyTorch Geometric (or DGL): constructing graph data objects, defining GNN layers, training loops.
- Credit risk basics: default, recovery rate, capital adequacy, leverage.
- Linear algebra: matrix multiplication, eigenvalues (for the spectral motivation behind GCN).